

Multi-Agent Research Assistant - Interview Preparation

Interview Questions & Model Answers

Project: Multi-Agent Research Assistant (FastAPI, LangGraph, LangChain, PostgreSQL, ChromaDB, Redis, Streamlit). Use this guide to prepare for technical screens, system design, and coding interviews tied to this repository.

1. Project Overview & Elevator Pitch

Q: Give a one-minute overview of this project.

A: Multi-Agent Research Assistant is a production-style autonomous research system. A user submits a complex question via Streamlit or REST API. FastAPI creates a persisted session in PostgreSQL (SQLite in dev), then LangGraph orchestrates five specialized agents: Planner decomposes the query; Researcher runs Tavily web search and ChromaDB RAG; Verifier cross-checks confidence; Summarizer condenses findings; Writer produces a cited markdown report and Mermaid diagram. Results are stored, exportable (MD/PDF), and streamable via SSE.

Q: What problem does it solve?

A: Single-shot LLM answers hallucinate and lack traceability. This system decomposes research, retrieves external evidence, verifies claims, cites sources, and persists full session history for audit and reuse.

Q: Who are the primary users?

A: Researchers, analysts, and engineers who need structured, sourced reports from open-ended questions-not just chat replies.

Q: What is in scope vs out of scope?

A: In scope: multi-step research, RAG on uploaded docs, session CRUD, streaming progress, demo/offline mode. Out of scope (typical): enterprise SSO, multi-tenant billing, fine-tuned domain models, real-time collaborative editing.

2. System Architecture & Design

Q: Describe the high-level architecture.

A: Layered: Streamlit UI -> FastAPI (routes, middleware, exception handlers) -> Service layer (ResearchService, DocumentService, SearchService) -> LangGraph workflow + RAG pipeline -> Repositories -> PostgreSQL, Redis, ChromaDB.

Q: Why separate routes, services, and repositories?

A: Routes handle HTTP and validation; services own business orchestration and workflow lifecycle; repositories isolate SQLAlchemy queries. This improves testability and keeps agents independent of transport.

Q: Explain the create_app() / lifespan pattern in FastAPI.

A: create_app() builds the ASGI app with CORS, middleware, and routers. lifespan initializes async DB engine (create_all), Redis pool, and Chroma client on startup, stores them on app.state, and disposes connections on shutdown-graceful degradation if Redis/Chroma fail.

Q: How does the frontend talk to the backend?

A: Streamlit uses httpx against BACKEND_URL (default localhost:8000). Research mode calls POST /api/v1/research and may consume SSE streams for live agent progress; chat mode uses search endpoints.

Q: What design patterns did you use?

A: Repository pattern, service layer, dependency injection (FastAPI Depends), factory (create_app), state machine (LangGraph StateGraph), template method (BaseAgent with shared logging/timing/error handling).

Q: How would you scale this horizontally?

Multi-Agent Research Assistant - Interview Preparation

A: Stateless API workers behind a load balancer; PostgreSQL + Redis as shared stores; Chroma as dedicated service; async job queue (Celery/RQ) for long workflows instead of blocking request threads; SSE via Redis pub/sub.

3. LangGraph & Multi-Agent Orchestration

Q: Why LangGraph instead of a simple LangChain chain?

A: Research needs conditional routing-retries after planner failure, re-research when verification confidence is low, early END when plan is empty. LangGraph models explicit nodes, edges, and shared state updates.

Q: What is ResearchState?

A: A TypedDict (total=False) passed between nodes: query, plan, research_results, verified_facts, citations, summary, report, mermaid_diagram, errors, metadata, retry_count, max_retries, session_id, status. Agents return partial updates.

Q: Walk through the workflow graph.

A: START -> planner -> (conditional) researcher -> verifier -> (conditional) summarizer -> writer -> END. Back-edges: planner error may skip to researcher; researcher retries on error; verifier may loop to researcher if confidence below threshold (~0.4).

Q: What does each agent do?

A: Planner: LLM decomposes query into sub-questions (demo plan offline). Researcher: Tavily + RAG per sub-question, synthesizes findings. Verifier: cross-reference, confidence scores, contradictions. Summarizer: executive summary. Writer: full markdown report + Mermaid diagram.

Q: How is retry logic implemented?

A: State fields retry_count and max_retries; conditional edge functions increment count and route back to researcher or END when exhausted.

Q: How do you inject mocks for testing agents?

A: ResearchWorkflow accepts optional agent instances in __init__; tests patch LLM clients or use demo_mode / use_offline_mode from Settings.

Q: What happens if the planner returns an empty plan?

A: Conditional routing ends the workflow early (no wasted API calls).

Q: How would you add a new agent (e.g., FactChecker)?

A: Implement BaseAgent subclass, add node to StateGraph, extend ResearchState if needed, wire conditional edges, persist step in StepRepository, update Streamlit viz (AGENT_PIPELINE).

4. FastAPI, API Design & Middleware

Q: Main API route groups?

A: /api/v1/research (start, list, get, delete, export, stream), /api/v1/documents (upload, stats, RAG ingestion), /api/v1/search (quick chat search), /health and root for liveness.

Q: How are requests validated?

A: Pydantic v2 schemas (ResearchRequest, etc.) at route boundaries; Settings validated via pydantic-settings from .env.

Q: How does SSE streaming work?

A: StreamingResponse with async generator yielding JSON events (StreamEvent) as workflow steps complete-frontend renders live progress.

Q: What does request_logging_middleware do?

Multi-Agent Research Assistant - Interview Preparation

A: structlog with correlation_id, method, path, duration_ms, status-supports debugging distributed-style flows in one process.

Q: How are domain errors mapped to HTTP?

A: BaseAppException subclasses with http_status; custom_exception_handler returns structured JSON {error_code, message, details}.

Q: Why async SQLAlchemy?

A: Non-blocking DB I/O under concurrent FastAPI requests; create_async_engine with asyncpg (Postgres) or aiosqlite (tests/dev).

5. RAG Pipeline & Vector Search

Q: Describe the RAG flow.

A: Document upload -> chunker (size/overlap from config) -> embeddings (OpenAI text-embedding-3-small) -> ChromaDB collection -> retriever returns top-k chunks for Researcher prompts.

Q: Why ChromaDB?

A: Purpose-built vector store, simple HTTP client, good for prototype-to-prod RAG without managing pgvector initially.

Q: How do you prevent context overflow?

A: Chunking with CHUNK_SIZE/CHUNK_OVERLAP; retriever limits k; agents summarize per sub-question before merging.

Q: What if Chroma is down?

A: lifespan logs chromadb_init_failed; app runs degraded-web search still works; document RAG unavailable until Chroma is up.

Q: How would you evaluate RAG quality?

A: Retrieval precision/recall on labeled Q&A, answer faithfulness metrics, human rubric on citations, A/B chunk sizes.

6. Data, Persistence & Caching

Q: What is stored in PostgreSQL?

A: ResearchSession (query, status, report, metadata) and ResearchStep records per agent execution for history and debugging.

Q: Role of Redis?

A: Caching and potential pub/sub for streaming; health check marks unhealthy if Redis unreachable but core flow can continue.

Q: Explain the repository pattern here.

A: SessionRepository and StepRepository encapsulate SQLAlchemy queries; services never write raw SQL in route handlers.

Q: How is pagination implemented for session list?

A: skip/limit query params on GET /api/v1/research; repository applies OFFSET/LIMIT.

7. Tools, Citations & Export

Q: How does Tavily integration work?

A: tavily_search / web_search tools wrap Tavily API with tenacity retries; Researcher merges web hits with RAG chunks.

Multi-Agent Research Assistant - Interview Preparation

Q: How are citations generated?

A: CitationExtractor matches sentences to sources via keyword overlap; APA formatting for bibliography; Writer embeds inline references.

Q: Export formats?

A: ReportExporter supports markdown and PDF (WeasyPrint) with metadata block.

Q: Mermaid diagrams-who generates them?

A: WriterAgent (LLM) or demo static diagram; DiagramGenerator utility for workflow/concept maps from structured step data.

8. Configuration, Security & Demo Mode

Q: How does demo_mode work?

A: Settings.demo_mode or missing real API keys triggers offline plans, mock search results, and canned reports-no paid API calls.

Q: Where are secrets loaded?

A: backend/.env via pydantic-settings; never committed; placeholders detected in has_openai / has_tavily helpers.

Q: Security concerns for production?

A: Add auth (JWT/OAuth), rate limiting, input sanitization, SSRF controls on URLs, secret rotation, CORS lockdown, RLS if multi-tenant Postgres.

Q: Why structlog?

A: Structured JSON logs, correlation IDs across middleware and agents, easier ingestion into ELK/Datadog.

9. Testing & Quality

Q: What tests exist?

A: pytest unit tests (agents, RAG, tools, workflow) and integration tests (health, research, documents) with httpx AsyncClient and mocked externals.

Q: Known test pitfalls in this codebase?

A: get_settings() is lru_cached-tests must clear cache or patch before import; AsyncClient should run app lifespan for DB init; tool tests must match API field names (label vs name).

Q: How would you test the LangGraph workflow?

A: Mock LLM responses per node, assert state transitions and conditional edges, snapshot final report structure.

10. DevOps & Docker

Q: docker-compose services?

A: backend, frontend, postgres, redis, chroma (8100->8000)-healthchecks and depends_on ensure ordered startup.

Q: Local dev without Docker?

A: pip install -r requirements.txt; SQLite DATABASE_URL override; DEMO_MODE; uvicorn backend.api.app:app; streamlit run frontend/app.py.

Q: Makefile targets you use most?

A: make dev, make frontend, make test, docker-up/down.

11. Behavioral & Scenario Questions

Multi-Agent Research Assistant - Interview Preparation

Q: Tell me about a hard bug you fixed.

A: (Example) Chroma port conflict with API on 8000-mapped Chroma to 8100 in compose; degraded health when Redis down but DB up.

Q: How do you trade off latency vs quality?

A: depth parameter controls plan size; max_retries caps re-research; streaming shows partial progress; optional skip verifier in future fast mode.

Q: How would you reduce OpenAI cost?

A: Smaller models for planner/summarizer, cache Tavily results in Redis, batch embeddings, truncate context, demo mode for dev.

12. Coding & Whiteboard Prompts

Q: Implement rate limiting on POST /research.

A: Redis sliding window or slowapi middleware keyed by client IP; return 429.

Q: Add parallel execution for sub-questions in Researcher.

A: asyncio.gather with semaphore bounded by MAX_CONCURRENT_AGENTS; merge results into research_results.

Q: Design API for canceling a running session.

A: POST /research/{id}/cancel sets flag in Redis; workflow nodes check flag between agents and raise WorkflowCancelled.

Q: SQL schema for sessions and steps (high level).

A: sessions: id UUID PK, query text, status enum, report text, timestamps; steps: id, session_id FK, agent_name, input/output JSON, duration_ms.